

STM32 EXTI peripheral

Ankit Srivastava

EE2028

ARMv7-M Architecture Ref Manual

RM0432 STM32L4+ Technical reference manual

PM0214 STM32 Cortex-M4 Programming Manual

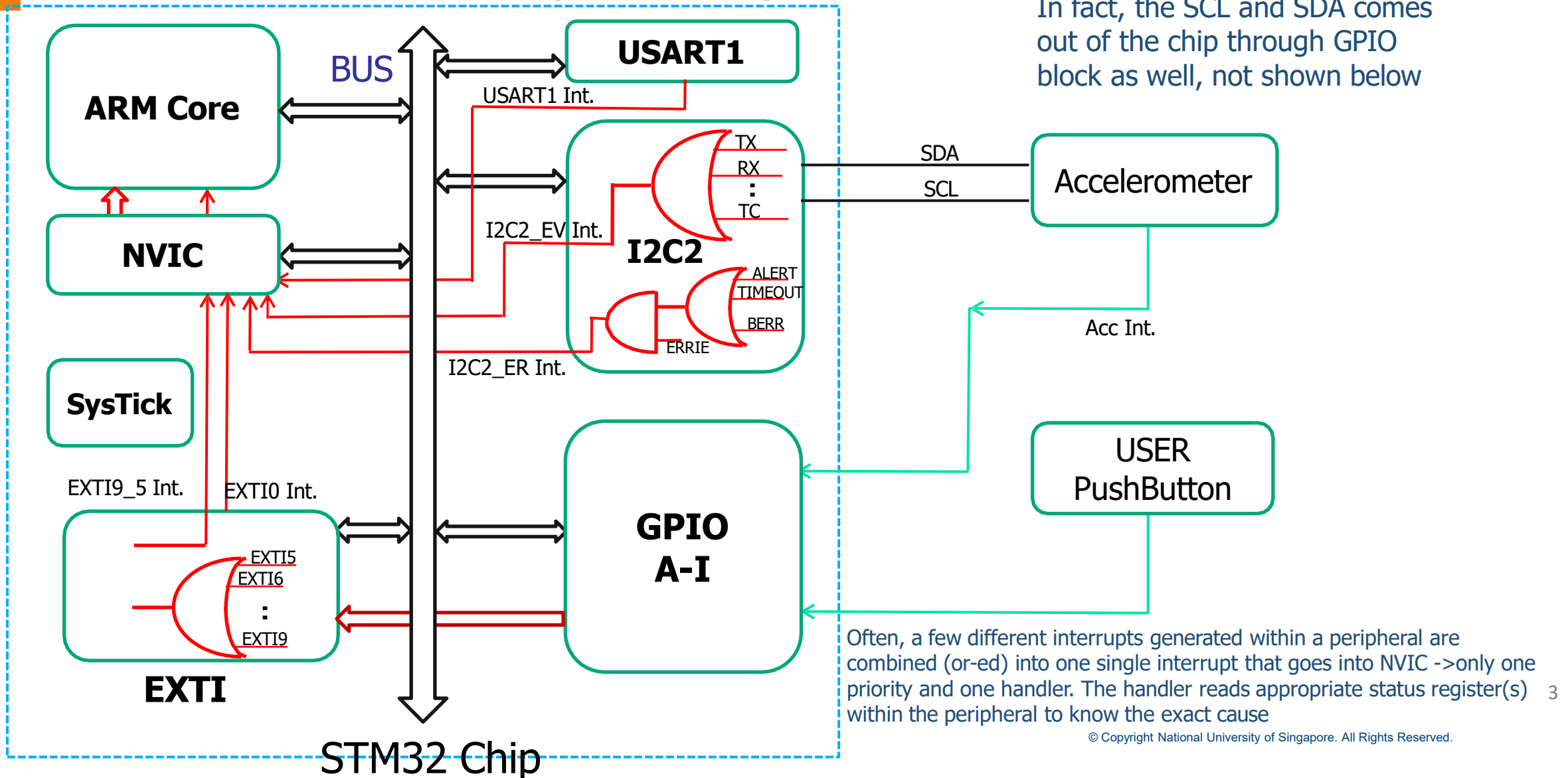
Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

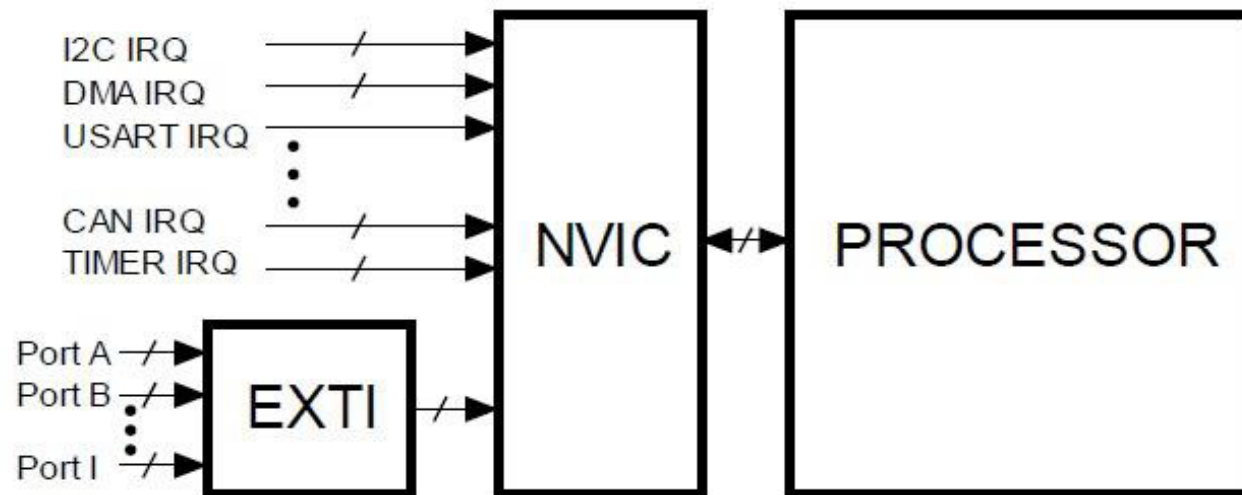
- EXTI Introduction
- EXTI Registers
- Configuration and Handling of EXTI

NVIC CONNECTIONS (PARTIAL)



EXTI INTRODUCTION

- The EXTI manages the external and internal asynchronous events/interrupts and generates the event request to NVIC
- Interrupts from other chips/devices (e.g., accelerometer, pressure sensor) cannot be connected directly to NVIC- they have to be forwarded/taken in through GPIO Pins => EXTI => NVIC
- The EXTI allows the management of up to 40 interrupt/event requests, including 26 configurable lines



EXTI INTRODUCTION

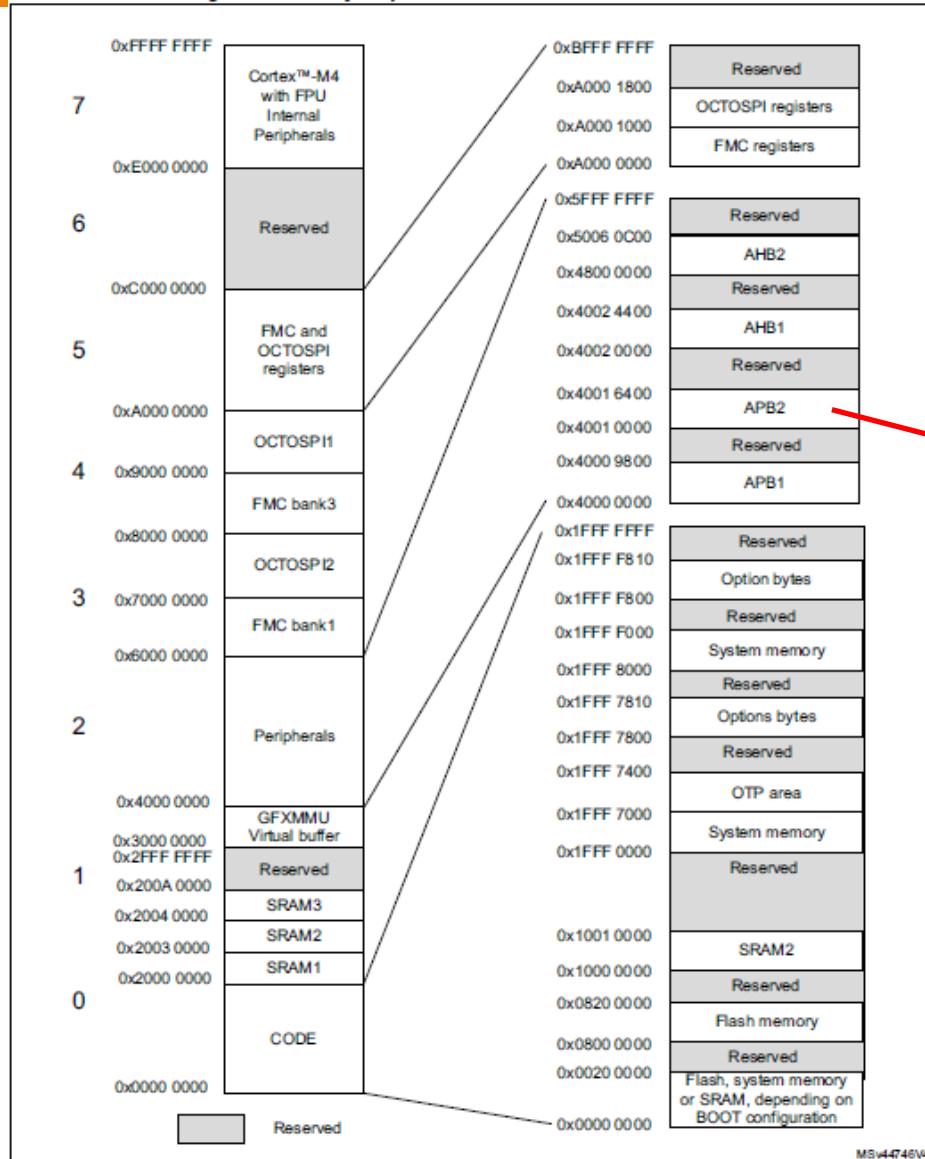
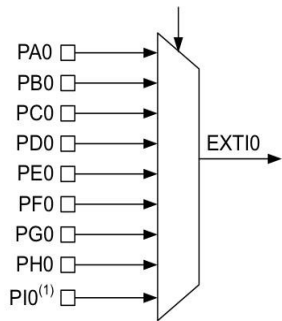


Table 1. STM32L4Rxxx and STM32L4Sxxx memory map and peripheral register boundary addresses (continued)

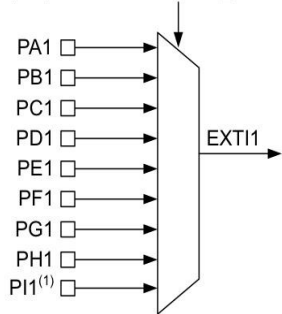
Bus	Boundary address	Size (bytes)	Peripheral	Peripheral register map
	0x4001 7400 - 0x4001 FFFF	33 KB	Reserved	-
	0x4001 6C00 - 0x4001 73FF	1 KB	DSIHOST	Section 30.15: DSI Host registers
	0x4001 6800 - 0x4001 6BFF	1 KB	LCD-TFT	Section 29.8.26: LTDC register map
	0x4001 6000 - 0x4001 67FF	2 KB	DFSDM1	Section 28.8.16: DFSDM register map
	0x4001 5C00 - 0x4001 5FFF	1 KB	Reserved	-
	0x4001 5800 - 0x4001 5BFF	1 KB	SAI2	Section 53.6.20: SAI register map
	0x4001 5400 - 0x4001 57FF	1 KB	SAI1	Section 53.6.20: SAI register map
	0x4001 4C00 - 0x4001 53FF	2 KB	Reserved	-
	0x4001 4800 - 0x4001 4BFF	1 KB	TIM17	Section 39.6.21: TIM16/TIM17 register map
	0x4001 4400 - 0x4001 47FF	1 KB	TIM16	Section 39.6.21: TIM16/TIM17 register map
	0x4001 4000 - 0x4001 43FF	1 KB	TIM15	Section 39.5.21: TIM15 register map
	0x4001 3C00 - 0x4001 3FFF	1 KB	Reserved	-
	0x4001 3800 - 0x4001 3BFF	1 KB	USART1	Section 50.8.15: USART register map
	0x4001 3400 - 0x4001 37FF	1 KB	TIM8	Section 37.4.33: TIM8 register map
	0x4001 3000 - 0x4001 33FF	1 KB	SPI1	Section 52.6.8: SPI register map
	0x4001 2C00 - 0x4001 2FFF	1 KB	TIM1	Section 37.4.32: TIM1 register map
	0x4001 2000 - 0x4001 2BFF	3 KB	Reserved	-
	0x4001 1C00 - 0x4001 1FFF	1 KB	FIREWALL	Section 4.4.8: Firewall register map
	0x4001 0800 - 0x4001 1BFF	5 KB	Reserved	-
	0x4001 0400 - 0x4001 07FF	1 KB	EXTI	Section 16.5.13: EXTI register map
	0x4001 0200 - 0x4001 03FF	1 KB	COMP	Section 26.6.3: COMP register map
	0x4001 0030 - 0x4001 01FF	1 KB	VREFBUF	Section 23.3.3: VREFBUF register map
	0x4001 0000 - 0x4001 002F	1 KB	SYSCFG	Section 9.2.12: SYSCFG register map

EXTI REGISTERS

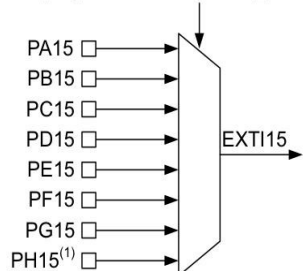
EXTI0[3:0] bits in the SYSCFG_EXTICR1 register



EXTI1[3:0] bits in the SYSCFG_EXTICR1 register



EXTI15[3:0] bits in the SYSCFG_EXTICR4 register



- Interrupts from GPIO pins are configurable, and occupying EXTI line 0-15.

Registers	Register	Address Offset	EXTI Lines
SYSCFG External Interrupt Configuration Registers	SYSCFG_EXTICR1	0x08	EXTI0-3
	SYSCFG_EXTICR2	0x0C	EXTI4-7
	SYSCFG_EXTICR3	0x10	EXTI8-11
	SYSCFG_EXTICR4	0x14	EXTI12-15

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Bits 31:16 Reserved, must be kept at reset value.
- Bits 15:0 bits are grouped in 4, used to choose the source for EXTI line x: 0b0000: PA[x], 0b0001: PB[x], 0b1000: PI[x]*

*Depends on the SoC, refer to STM32L4S5xx datasheet

EXTI REGISTER: EXTI_IMR1

- Interrupt mask registers: Address Offset = 0x00 (n=1) : RW
- Bits 31:0 IMx: Interrupt Mask on line x (x=31 to 0)
 - 0: Interrupt request from Line x is masked, i.e., disabled (default for EXTI0-15)
 - 1: Interrupt request from Line x is not masked
- EXTI line 0-15: GPIO configurable lines. We do not care about bits 31:16 while configuring GPIO EXTI lines

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IM31	IM30	IM29	IM28	IM27	IM26	IM25	IM24	IM23	IM22	IM21	IM20	IM19	IM18	IM17	IM16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IM15	IM14	IM13	IM12	IM11	IM10	IM9	IM8	IM7	IM6	IM5	IM4	IM3	IM2	IM1	IM0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

EXTI REGISTER: EXTI_RTSR1

- Rising trigger selection registers: Address Offset = 0x08 (n=1) : RW
- Bits 31:23 and 17: Reserved, must be kept at reset value
- Bits 22:18 and 16:0 RTx: Rising trigger event configuration bit of line x (x=22 to 18, and 16 to 0). We do not care about bits 31:16 while configuring GPIO EXTI lines
 - 0: Rising trigger disabled for input line
 - 1: Rising trigger enabled for input line

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RT22	RT21	RT20	RT19	RT18	Res.	RT16
									rw	rw	rw	rw	rw		rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RT15	RT14	RT13	RT12	RT11	RT10	RT9	RT8	RT7	RT6	RT5	RT4	RT3	RT2	RT1	RT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

EXTI REGISTER: EXTI_FTSR1

- Falling trigger selection registers: Address Offset = 0x0C (n=1) : RW
- Bits 31:23 and 17: Reserved, must be kept at reset value
- Bits 22:18 and 16:0 FTx: Falling trigger event configuration bit of line x (x=22 to 18, and 16 to 0). We do not care about bits 31:16 while configuring GPIO EXTI lines
 - 0: Falling trigger disabled for input line
 - 1: Falling trigger enabled for input line
- EXTI_RTSTR and EXTI_FTSR can be set for the same interrupt lines

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FT22	FT21	FT20	FT19	FT18	Res.	FT16
									rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FT15	FT14	FT13	FT12	FT11	FT10	FT9	FT8	FT7	FT6	FT5	FT4	FT3	FT2	FT1	FT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

EXTI REGISTER: EXTI_RTSR1 AND EXTI_FTSR1

- Is it possible to have both rising edge and falling edge enabled for the same EXTI?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RT22	RT21	RT20	RT19	RT18	Res.	RT16
									rw	rw	rw	rw	rw		rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RT15	RT14	RT13	RT12	RT11	RT10	RT9	RT8	RT7	RT6	RT5	RT4	RT3	RT2	RT1	RT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FT22	FT21	FT20	FT19	FT18	Res.	FT16
									rw	rw	rw	rw	rw		rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FT15	FT14	FT13	FT12	FT11	FT10	FT9	FT8	FT7	FT6	FT5	FT4	FT3	FT2	FT1	FT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

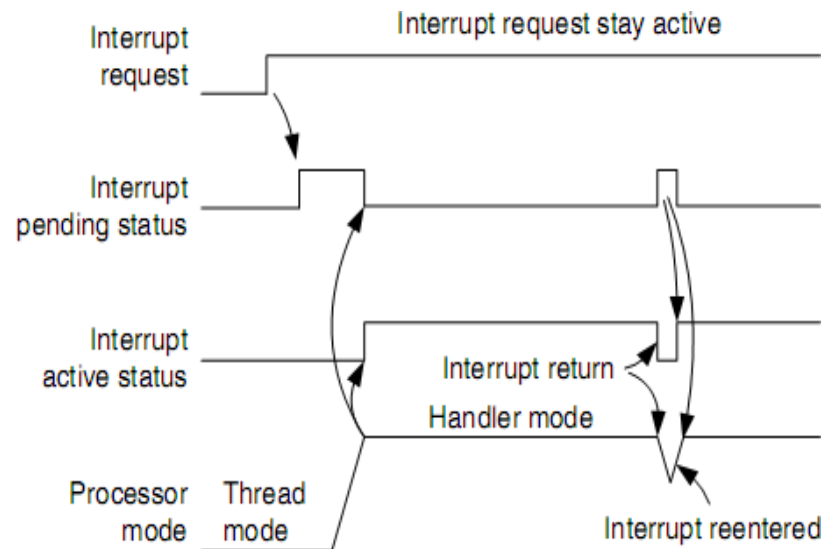
EXTI REGISTER: EXTI_PR1

- Pending registers: Address Offset = 0x14 (n=1) : rc_w1 (read/write 1 to clear)
- Bits 31:23 and 17: Reserved, must be kept at reset value
- Bits 22:18 and 16:0 PIFx: Pending interrupt flag on line x (x=22 to 18, and 16 to 0).
We do not care about bits 31:16 while configuring GPIO EXTI
 - 0: No trigger request occurred
 - 1: Selected trigger request occurred
- This bit is set when the selected edge event arrives on the interrupt line.
- This bit is cleared by writing a '1' to the bit

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PIF22	PIF21	PIF20	PIF19	PIF18	Res.	PIF16
									rc_w1	rc_w1	rc_w1	rc_w1	rc_w1		rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIF15	PIF14	PIF13	PIF12	PIF11	PIF10	PIF9	PIF8	PIF7	PIF6	PIF5	PIF4	PIF3	PIF2	PIF1	PIF0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

EXTI REGISTER: EXTI_PR1

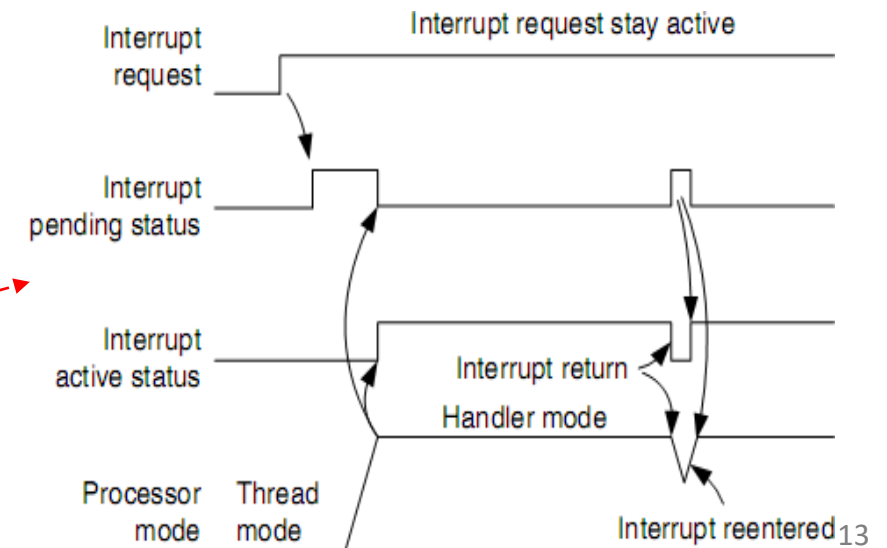
- What if you forget to clear the pending bit?



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PIF22	PIF21	PIF20	PIF19	PIF18	Res.	PIF16
									rc_w1	rc_w1	rc_w1	rc_w1	rc_w1		rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PIF15	PIF14	PIF13	PIF12	PIF11	PIF10	PIF9	PIF8	PIF7	PIF6	PIF5	PIF4	PIF3	PIF2	PIF1	PIF0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

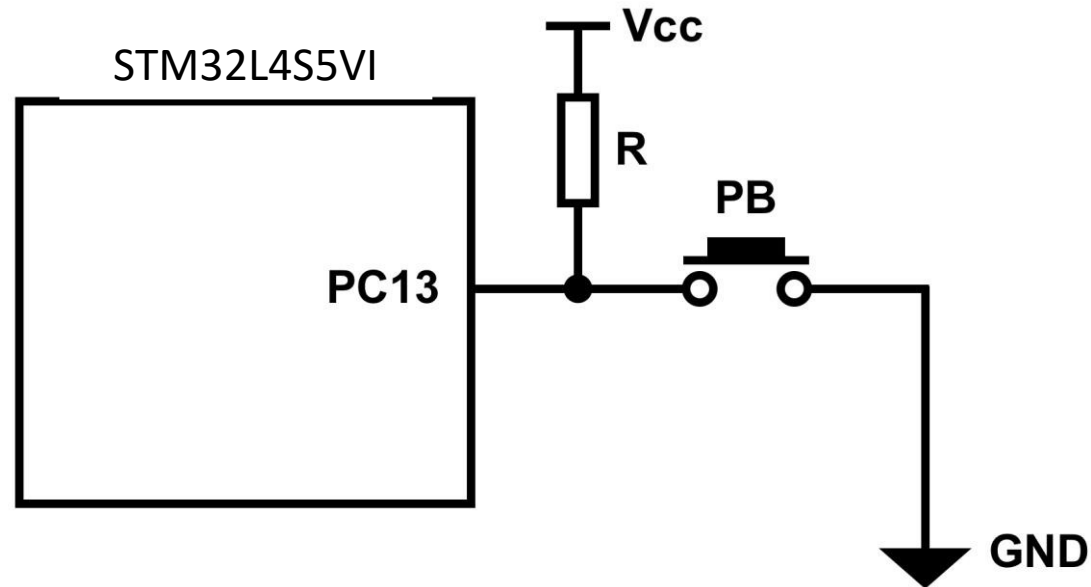
CONFIGURATION AND HANDLING OF EXTI

- Configuration steps:
 - Enable AHB2 bus for the GPIO port(s)
 - Configure SYSCFG_EXTICRn to choose Port Pin as source of the specific EXTI line
 - Configure EXTI_IMRn to enable the specific EXTI line
 - Configure EXTI_RTSRn or/and EXTI_FTSRn to choose the Rising or Falling edge (or both) trigger
 - Enable specific interrupt request (IRQ) in NVIC
- Once the settings are done, the user program need not 'call' the handler, it will be invoked by the hardware automatically
- At the end of handler, should clear the interrupt flag by writing to the pending register EXTI_PRn
 - what if you don't?



EXAMPLE

- A pushbutton connected to STM32L4S5VI port-C pin-13 is required to be configured as EXTI interrupt input, so that the processor is able to print another message when pushbutton is pressed



- Falling/rising edge should be used depending on the connected hardware. Active low pushbutton = falling edge

EXAMPLE

```
#include "main.h"

static void MX_GPIO_Init(void);
extern void initialise_monitor_handles(void);
void SystemClock_Config(void);

HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == BUTTON_EXTI13_Pin)
    {
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14);
        printf("\t Blue button is pressed. \n");
    }
}

int main(void)
{
    initialise_monitor_handles();
    HAL_Init();
    MX_GPIO_Init();

    while (1)
    {
        printf("P\n");
        printf("O\n");
        printf("L\n");
        printf("L\n");
        printf("I\n");
        printf("N\n");
        printf("G\n\n");
    }
}
```

Called by HAL_GPIO_EXTI_IRQHandler(),
which in turn is called by
EXTI15_10_IRQHandler()
HAL_GPIO_EXTI_IRQHandler() checks if
EXTI_PR1[13] = 0b1 and clears it

```
static void MX_GPIO_Init(void)
{
    __HAL_RCC_GPIOB_CLK_ENABLE(); // Enable AHB2 Bus for GPIOB
    __HAL_RCC_GPIOC_CLK_ENABLE(); // Enable AHB2 Bus for GPIOC

    HAL_GPIO_WritePin(GPIOB, LED2_Pin, GPIO_PIN_RESET); // Reset the LED2_Pin as 0

    GPIO_InitTypeDef GPIO_InitStruct = {0};

    // Configuration of LED2_Pin (GPIO-B Pin-14) as GPIO output
    GPIO_InitStruct.Pin = LED2_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    // Configuration of BUTTON_EXTI13_Pin (GPIO-C Pin-13) as AF,
    GPIO_InitStruct.Pin = BUTTON_EXTI13_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    // Enable NVIC EXTI line 13
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
}
```

EXTI10 to EXTI15 is OR-ed together
and sent as a single interrupt to NVIC.
This means the same priority and the
same handler for these EXTIs

Configures SYSCFG_EXTICR4[7:4] = 0b0010,
EXTI_IMR1[13]=0b1, EXTI_FTSR1[13]=0b1

CONCLUSION

- EXTI Introduction
- EXTI Registers
 - SYSCFG
 - IMR
 - RTSR
 - FTSR
 - PR
- Configuration and Handling of EXTI
 - `demo_exti_print` example code